

DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	BIJIVEMULA SHANMUKHASAI
ROLL NO	AV.SC.U4CSE25021
YEAR/SEM/SECTION	2nd YEAR /SEM-3/CSE A
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO	DATE
1.	**WEEK-1**:1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	3-12	23-07-26
2.	**WEEK-2**: 2a. Python program to perform linear search on the given input. 2b. Python program to perform binary search on the given input. 2c. Verify by taking sorted array and unsorted array as an input.	13-18	30-07-26
3.	**WEEK -3**: 3a. Python program to perform bubble sort on the given input. 3b. Python program to perform insertion sort on the given input. 3c. Python program to perform selection sort on the given input. 3d. Python program to perform merge sort on the given input. 3e. Python program to perform quick sort on the given input.	19-27	06-08-26
4.	**WEEK-4:** 4a. Write the python program to implement a single linked list which could perform the below operations,by taking an input from the user. 4b. Write the python program to implement a double linked list which could perform the below operations,by taking an input from the user. 4c. Write the python program to implement a circular linked list which could perform the below operations,by taking an input from the user.	28-46	13-08-26

5.	**WEEK-5**:Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements	47-51	27-08-26
6.	**WEEK-6**: 6a. Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using array, linked list. 6b. Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using circular linked list.	52-59	03-09-26
7.	**WEEK-7**: 7. Write a program to implement a binary tree using an array and linked list with all of its operation.	60-67	24-09-26
8.	**WEEK-8:** 8. Write a python program to implement Binary Search Trees with its operations.	68-72	08-10-26
9.	**WEEK-9: 9.** Write a python program to implement AVL trees with its operation.	73-78	22-10-26
10.	**WEEK-10:** 10**.** Write a python program to Implement Hashing Using Separate Chaining, Linear Probing and Quadratic Probing.	79-82	29-10-26

LAB -1

1)AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "launch".

PROGRAM:

```
def launch(n):
    for i in range(n,-1,-1):
        if i==0:
print("launch"):
        else:
            print(i)
n=int (input ("enter no. of seconds to launch:"))
launch(n)
```

OUTPUT:

```
>>
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/lab 1
.py
enter no. of seconds to launch:5
5
4
3
2
1
launch
>>
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Indentation Error	Incorrect indentation	Align blocks properly
Name Error	Variable not defined	Define before use
Value Error	Non-integer input	Enter a valid integer

2)AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers. This can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

PROGRAM:

```
def intrest(p,n):
    if n==0:
        return 1
    else:
        return p*intrest(p,n-1)
n=int(input("enter no.of years:"))
p=int(input("enter a number:"))
res=intrest(p,n)
print(res)
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/lab 1
.py
enter no.of years:5
enter a number:4
1024
```

EXPLANATION:

This program recursively calculates p^n by multiplying p until n becomes 0.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Recursion error	Very large value of n	Reduce input
Name Error	Wrong function/variable name	Check spelling
Value Error	Non-integer input	Enter a valid integer

3)AIM: An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(m)
    key=m
    for i in range(n-1):
        if arr[i]==key:
            return i
n=int(input("enter no.of employees to add:"))
arr=[]
for i in range(n):
    a=int(input("enter a value:"))
    arr.append(a)
print(arr)
m=int(input("enter a element to search:"))
```

```
result=search(m)
if result==-1:
    print(“element is not found”)
else:
    print("employee id is stored at the index:",result)
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/lab 1
.py
Enter no. of employees to add: 7
Enter a value: 12
Enter a value: 23
Enter a value: 45
Enter a value: 67
Enter a value: 78
Enter a value: 89
Enter a value: 90
[12, 23, 45, 67, 78, 89, 90]
Enter an element to search: 45
Employee ID is stored at the index: 2
```

EXPLANATION:

This program searches for an employee ID in a list and returns its index if found.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Syntax error	Wrong quotes/brackets	Correct syntax
Logic error	Returns None instead of -1	Return -1 when not found

4)AIM: In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula(n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def fact(n):
    fact=1
    for i in range(1,n+1):
        fact=fact*i

    return fact
n=int(input("enter a number to find its factorial:"))
result=fact(n)
print(result)
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/lab 1
.py
enter a number to find its factorial:5
120
```

EXPLANATION:

This program calculates the factorial of a number using iteration (not recursion).

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Overflow/large input	Large input value	Give smaller input
Value error	Invalid input	Even integer

5)AIM: Write a recursive Python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fib(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return fib(n-1)+fib(n-2)


n=int(input("enter no of terms:"))
if n<=-1:
    print("invalid input")
else:
    for i in range(n):
        print(fib(i),end=" ")
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/lab 1
.py
enter no of terms:6
0 1 1 2 3 5
```

EXPLANATION:

The program uses recursion to generate Fibonacci numbers. fib (0)=0 and fib (1) =1 are the base cases. For every other term, fib(n)=fib(n-1) + fib(n-2). The loop prints the first n terms.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Recursion error	Large value of n	Use iteration
Invalid input	Negative value of n	Validate input
Slow execution	Repeated recursive calls	Use dynamic programming

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1. countdown to launch	**O(n)**	**O (1)**	The program uses a loop that prints from **n to 0** once.
2. Power calculation	**O(n)**	**O(n)**	One recursive call is made for each value of **n**—the recursion stack stores **n** calls.
3. Employee ID search	**O(n)**	**O (1)**	The program performs a linear search through the list.
4. Factorial	**O(n)**	**O (1)**	The program calculates factorial using a loop from **1 to n**.
5. Fibonacci series	**$O(2^n)$**	**O(n)**	Each function call creates two additional recursive calls, leading to exponential time complexity. The recursion stack reaches depth **n**.

Lab 2

1. AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i]==key:
            return i
    return -1


n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key=int(input("Enter element to search:"))
result=linear_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/p
ratice.py
Enter the number of elements: 5
Enter the elements:
1
2
3
4
5
Enter the element to search: 4
Element found at position 4
```

EXPLANATION:

- The linear_search() function checks each element of the list one by one and compares it with the given `key`.
- If the key is found, its index is returned; otherwise, the function returns `-1`.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2. AIM: Write a Python program to perform binary search on the given input. Verify by taking a sorted array and an unsorted array as input.

PROGRAM:

```
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return mid
        elif arr[mid]<key:
            low=mid+1
        else:
```

```
            high=mid-1
    return -1


n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/p
ratice.py
Enter the number of elements: 5
Enter the elements separated by space: 1 3 5 7 9
Enter the element to search: 7
The given array is already sorted.
Element 7 found at position 4 in the sorted array.
```

3. PROGRAM:

```
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
```

```
        if arr[mid]==key:
            return mid
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid-1
    return -1

n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
 if arr==sorted(arr):
    print("\n The input list is already sorted.")
else:
    print("\n The input list is not sorted.")
    print("Sorting the list...")
    arr.sort()
print("Sorted list.",arr)
 key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/p
ratice.py
Enter no. of elements: 5
Enter the elements:
1
4
6
8
10

The input list is already sorted.
Sorted list: [1, 4, 6, 8, 10]
Enter element to search: 3
Element not found.
```

EXPLANATION:

- The binary_search() function repeatedly divides the sorted list into two halves and compares the middle element with the search key.
- If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.
- The second program verifies whether the input is sorted and sorts it using arr.sort() if necessary before performing binary search.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Binary search may give an incorrect result if the input array is unsorted.	Check whether the array is sorted and use arr.sort() before calling binary_search().
2.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialise low = 0, high = len(arr) - 1, and update them as low = mid + 1 or high = mid - 1.

LAB -3

AIM: 3a. Write a Python program to sort the elements using a bubble sort technique. Take the input elements from the user.

Program:

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] < arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Elements after Bubble Sort:")
print(arr)
```

Output:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/LAB 3
.py
Enter the number of elements: 5
Enter the elements:
12
23
45
67
89
Elements after Bubble Sort:
[89, 67, 45, 23, 12]
```

Explanation:

• n = int(input(...)) – Gets the number of elements.

• arr = [] – Creates an empty list to store elements.

• for i in range(n) – Takes n elements as input.

• arr.append(...) – Adds each element to arr.

• for i in range(n - 1) – Performs Bubble Sort passes.

• for j in range(n - i - 1) – Compares adjacent elements.

• if arr[j] < arr[j + 1] – Checks if the current element is smaller.

• arr[j], arr[j + 1] = ... – Swaps the elements to sort in descending order.

• print(arr) – Displays the sorted list.

Error Table:

S.NO	Error Obtained	Rectification
1.	Sorting Order Error – Using > instead of < in the if condition changes the order to ascending.	Use if arr[j] < arr[j + 1]: for descending order.

AIM: 3b.Write a python program to sort the elements using an insertion sort technique. Take the input elements from the user.

Program:

```
def insertion_sort(arr):
    n = len(arr)
    for i in range (1,n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j = j - 1
        arr[j+1] = key
    return arr
```

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    element = int(input())
    arr.append(element)
result = insertion_sort(arr)
print("Elements after Insertion Sort:")
print(result)
```

Output:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/LAB 3
.py
Enter the number of elements: 3
Enter the elements:
23
27
12
Elements after Insertion Sort:
[12, 23, 27]
```

Explanation:

• n = len(arr) – Gets the list size.

• for i in range(1, n) – Selects elements from the second position.

• key = arr[i] – Stores the current element.

• j = i - 1 – Sets the previous index.

• while j >= 0 and arr[j] > key – Compares previous elements with key.

• arr[j + 1] = arr[j] – Shifts larger elements right.

• arr[j + 1] = key – Inserts key in the correct position.

• return arr – Returns the sorted list.

• result = insertion_sort(arr) – Calls the function.

• print(result) – Displays the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Insertion Error – Placing key at the wrong index.	Use arr[j + 1] = key after shifting elements.
2.	Function Call Error – Not calling insertion_sort(arr) correctly.	Pass the input list arr to the function.

AIM: 3c.Write a python program to sort the elements using a selection sort technique. Take the input elements from the user.

Program:

```
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range (i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
        arr[i],arr[min_index]=arr[min_index],arr[i]
    return arr
n=int(input("Enter the number of elements: "))
arr=[]
print("Enter the elements:")
for i in range(n):
    element=int(input())
    arr.append(element)
```

result=selection_sort(arr)

print("Elements after Selection Sort:")

print(result)

Output:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/LAB 3
.py
Enter the number of elements: 4
Enter the elements:
2
3
6
10
Elements after Selection Sort:
[2, 3, 6, 10]
```

Explanation:

• def selection_sort(arr) – Defines the Selection Sort function.

• n = len(arr) – Gets the number of elements in arr.

• for i in range(n-1) – Performs sorting passes.

• min_index = i – Assumes the first element is the smallest.

• for j in range(i+1,n) – Checks the remaining elements.

• if arr[j] < arr[min_index] – Finds the smallest element.

• arr[i], arr[min_index] = ... – Swaps the smallest element into position.

• return arr – Returns the sorted list.

• result = selection_sort(arr) – Calls the function.

• print(result) – Displays the sorted list in ascending order.

Error Table:

S.NO	Error Obtained	Rectification
1.	Swap Error – Incorrectly swapping the elements.	Use arr[i], arr[min_index] = arr[min_index], arr[i].
2.	Minimum Index Error – Incorrectly assigning min_index.	Use min_index = i.

AIM: 3d.Write a python program to sort the elements using a merge sort technique. Take the input elements from the user.

Program:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid=len(arr)//2
    left=merge_sort(arr[:mid])
    right=merge_sort(arr[mid:])
    i = j = k = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1
```

```
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
    return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    element = int(input())
    arr.append(element)
result = merge_sort(arr)
print("Elements after Merge Sort:")
print(result)
```

Output:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/LAB 3
.py
Enter the number of elements: 5
Enter the elements:
12
23
67
90
43
Elements after Merge Sort:
[12, 23, 43, 67, 90]
```

Explanation:

• def merge_sort(arr) – Defines the Merge Sort function.

• if len(arr) <= 1 – Stops when the list has one or no elements.

• mid = len(arr)//2 – Finds the middle of the list.

• left and right – Divide the list into two halves recursively.

• i = j = k = 0 – Initializes the indexes.

• while loops – Compare and merge the elements.

• if left[i] <= right[j] – Selects the smaller element.

• arr[k] – Places the selected element in the list.

Error Table:

S.NO	Error Obtained	Rectification
1.	Index Error – Incorrectly updating i, j, or k.	Increment the appropriate index after each insertion.
2.	Recursion Error – Incorrectly dividing the list.	Use arr[:mid] and arr[mid:].

AIM: 3e.Write a python program to sort the elements using a quick sort technique. Take the input elements from the user.

Program:

```
def quicksort(a,low,high):
    if low < high:
        i = low
        j = high
        pivot = low
        while i < j:
            while i < len(a) and a[i]<=a[pivot]:
                i +=1
```

```
        while a[j] > a[pivot]:
            j = j -1
        if i < j:
            a[i],a[j] = a[j],a[i]
    a[j],a[pivot] = a[pivot],a[j]
    quicksort(a,low,j-1)
    quicksort(a,j+1,high)
a = list(map(int,input("Enter numbers to sort:").split(" ")))
n = len(a)
quicksort(a,0,n-1)
print("Sorted array:")
print(a)
```

Output:

```
= RESTART: C:/Users/bijiv/OneDrive/Documents/AV.SC.U4CSE25021/LAB 3
.py
Enter numbers to sort:3
Sorted array:
[3]
```

Explanation:

• def quicksort(a, low, high) – Defines the Quick Sort function.

• if low < high – Checks the sorting range.

• i and j – Set left and right positions.

• pivot = low – Selects the first element as pivot.

• while i < j – Performs partitioning.

• Inner while loops – Find elements for swapping.

• a[i], a[j] = ... – Swaps elements.

• a[j], a[pivot] = ... – Places the pivot correctly.

• quicksort() calls – Sort the two partitions.

• input().split() – Takes space-separated numbers.

• print(a) – Displays the sorted array.

Error Table:

S.NO	Error Obtained	Rectification
1.	Pivot Error – Incorrectly selecting the pivot index.	Use pivot = low.
2.	Swap Error – Incorrect swapping of a[i] and a[j].	Use a[i], a[j] = a[j], a[i].

WEEK-4

AIM: 4a. Write the Python program to implement a single linked list which could perform the following operations by taking input from the user.

Program:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def deleteAtBeg(self):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted Value =", temp.data)

def deleteAtEnd(self):
    if self.head is None:
        print("No Data to delete")

    elif self.head.next is None:
        print("Deleted Value =", self.head.data)
        self.head = None

    else:
        temp = self.head

        while temp.next.next:
            temp = temp.next

        print("Deleted Value =", temp.next.data)
        temp.next = None

def delete(self, value):
    if self.head is None:
        print("No Data to delete")
        return

    temp = self.head

    if temp.data == value:
        self.head = temp.next
        print("Value deleted")
        return

    while temp.next and temp.next.data != value:
        temp = temp.next

    if temp.next is None:
        print("Value not present")
    else:
        temp.next = temp.next.next
        print("Value deleted")
```

```
    def count(self):
        if self.head is None:
            print("No Linked List")
        else:
            c = 0
            temp = self.head

            while temp:
                c += 1
                temp = temp.next

            print(f"Number of nodes = {c}")

    def display(self):
        if self.head is None:
            print("No Linked List")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")
```

```
ll = LinkedList()

while True:
    print("\n--- Singly Linked List ---")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete at Beginning")
    print("4. Delete at End")
    print("5. Delete a Value")
    print("6. Count Nodes")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        ll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        ll.insert_end(data)

    elif choice == 3:
        ll.deleteAtBeg()

    elif choice == 4:
        ll.deleteAtEnd()

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        ll.delete(value)

    elif choice == 6:
        ll.count()

    elif choice == 7:
        ll.display()

    elif choice == 8:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

Output:

```
>>
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab4.py
 SINGLY LINKED LIST
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 3
List is empty.
 SINGLY LINKED LIST
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
List is empty.
```

```
--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 7
10 -> 20 -> 30 -> None

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 6
Number of nodes = 3

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 3
Deleted Value = 10
```

```
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 7
20 -> None

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 5
Enter value to delete: 20
Value deleted

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 7
No Linked List

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 8
Program ended
```

Explanation:

This program implements a singly linked list in Python. Each node contains two parts: data and a next pointer that stores the address of the next node.

- Node class: Creates a node with data and next.
- insert_begin(): Inserts a new node at the beginning of the linked list.
- insert_end(): Inserts a new node at the end of the linked list.
- deleteAtBeg(): Deletes the first node from the linked list.
- deleteAtEnd(): Deletes the last node from the linked list.
- delete(value): Searches for a given value and deletes that node.
- count(): Counts and displays the total number of nodes.
- display(): Displays all node values from beginning to end.
- Main program: Uses a menu so the user can select different linked-list operations until choosing Exit.

Error Table:

S.NO	Error Obtained	Rectification
1.	Error while deleting from an empty list	Check if self.head is None before deletion.
2.	Last node was not deleted correctly	Handle the single-node case and set the previous node's next to None.
3.	Incorrect output while displaying the list	Traverse using while temp: and print each node's data.

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1. Insert at Beginning	**O(1)**	**O(1)**	Directly changes the head.
2.Insert at End	**O(n)**	**O(1)**	Traverse to the last node
3.Delete at Beginning	**O(1)**	**O(1)**	Directly removes the head
4.Delete at End	**O(n)**	**O(1)**	Traverse to the last node
5.Delete by value	**O(n)**	**O(1)**	May search the entire list
6.Count Nodes	**O(n)**	**O(1)**	Traverses all nodes
7.Display	**O(n)**	**O(1)**	Traverse all nodes

AIM: 4b. Write a Python program to implement a doubly linked list that performs the operations below by taking input from the user.

Program:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node

    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp
```

```python
  def insert_at_position(self, data, position):
      new_node = Node(data)

      if position == 1:
          self.insert_begin(data)
          return

      temp = self.head
      count = 1

      while temp is not None and count < position - 1:
          temp = temp.next
          count += 1

      if temp is None:
          print("Invalid position")
          return

      new_node.next = temp.next
      new_node.prev = temp

      if temp.next is not None:
          temp.next.prev = new_node

      temp.next = new_node

  def delete_begin(self):
      if self.head is None:
          print("List is empty")
          return

      self.head = self.head.next

      if self.head is not None:
          self.head.prev = None
```

```
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    if temp.next is None:
        self.head = None
        return

    while temp.next is not None:
        temp = temp.next

    temp.prev.next = None

def delete_at_position(self, position):
    if self.head is None:
        print("List is empty")
        return

    if position == 1:
        self.delete_begin()
        return

    temp = self.head
    count = 1

    while temp is not None and count < position:
        temp = temp.next
        count += 1

    if temp is None:
        print("Invalid position")
        return

    if temp.next is not None:
        temp.next.prev = temp.prev

    if temp.prev is not None:
        temp.prev.next = temp.next


def count(self):
    temp = self.head
    count = 0

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    print("Doubly Linked List:", end=" ")

    while temp is not None:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("END")
```

```python
dll = DoublyLinkedList()
while True:
    print("\n--- DOUBLY LINKED LIST ---")
    print("1. Insert Begin")
    print("2. Insert End")
    print("3. Insert at Specific Position")
    print("4. Delete Begin")
    print("5. Delete End")
    print("6. Delete at Specific Position")
    print("7. Count")
    print("8. Display")
    print("9. End")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        dll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        dll.insert_at_position(data, position)

    elif choice == 4:
        dll.delete_begin()

    elif choice == 5:
        dll.delete_end()

    elif choice == 6:
        position = int(input("Enter position: "))
        dll.delete_at_position(position)

    elif choice == 7:
        dll.count()

    elif choice == 8:
        dll.display()

    elif choice == 9:
        print("Program ended")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab4.py

--- Doubly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 5
Enter value to delete: 3
No Data to delete

--- Doubly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 2
Enter data: 1
```

```
--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 7
Number of nodes: 3

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 6
Enter position: 2

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 2
Enter data: 30

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 3
Enter data: 20
Enter position: 2

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 8
Doubly Linked List: 10 <-> 20 <-> 30 <-> END
```

```
--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 9
Program ended
```

Explanation of Code:

A **doubly linked list** is a collection of nodes where each node contains data, previous link (prev) and next link (next). The program uses a menu to perform insertion, deletion, counting, and display operations.

- **Node:** Stores data, prev, and next.
- **Head:** Points to the first node of the list.
- **Insert Begin:** Adds a new node at the beginning and updates the links.
- **Insert End:** Traverses to the last node and adds the new node.
- **Insert at Position:** Adds a node at the specified position by updating both links.
- **Delete Begin:** Removes the first node and changes the head.
- **Delete End:** Removes the last node and sets the previous node's next to None.
- **Delete at Position:** Removes the specified node and connects its previous and next nodes.
- **Count:** Traverses the list and counts the number of nodes.
- **Display:** Traverses and prints all node values.
- **End:** Stops the program using break.

Error Table:

S.NO	Error Obtained	Rectification
1.	Forgetting to update prev and next links.	Update both links correctly during insertion and deletion.
2.	Deleting a node from an empty list.	Check if self.head is None before deletion.
3.	Entering an invalid position.	Check the position and display “Invalid position”.

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1. Insert at Beginning	**O(1)**	**O(1)**	Directly inserts before head
2.Insert at End	**O(n)**	**O(1)**	Traverse the list to reach the last node.
3.Delete at Beginning	**O(1)**	**O(1)**	Directly removes the first node
4.Delete at End	**O(n)**	**O(1)**	Traverse to the last node
5.Delete by value	**O(n)**	**O(1)**	Traverses to the specified position
6.Count Nodes	**O(n)**	**O(1)**	Visits every node to count them

7. Display	**O(n)**	**O(1)**	Visits every node to display the data

AIM: 4c. Write the Python program to implement a circular linked list which could perform the below operations, by taking an input from the user.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_begin(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head
```

```
        -

    # Insert at specific position
    def insert_at_position(self, data, position):
        if position == 1:
            self.insert_begin(data)
            return

        new_node = Node(data)
        temp = self.head
        count = 1

        while temp.next != self.head and count < position - 1:
            temp = temp.next
            count += 1

        if count != position - 1:
            print("Invalid position")
            return

        new_node.next = temp.next
        temp.next = new_node


# Delete beginning
def delete_begin(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    self.head = self.head.next
    temp.next = self.head

# Delete end
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        return

    temp = self.head

    while temp.next.next != self.head:
        temp = temp.next

    temp.next = self.head
```

```
# Delete at specific position
def delete_at_position(self, position):
    if self.head is None:
        print("List is empty")
        return

    if position == 1:
        self.delete_begin()
        return

    temp = self.head
    count = 1

    while temp.next != self.head and count < position - 1:
        temp = temp.next
        count += 1

    if temp.next == self.head:
        print("Invalid position")
        return

    temp.next = temp.next.next

# Count
def count(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    temp = self.head
    count = 0

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes:", count)


 # Display
 def display(self):
     if self.head is None:
         print("List is empty")
         return

     temp = self.head

     print("Circular Linked List:", end=" ")

     while True:
         print(temp.data, end=" -> ")
         temp = temp.next

         if temp == self.head:
             break

     print("HEAD")
```

```
# Main program
cll = CircularLinkedList()
while True:
    print("\n--- CIRCULAR LINKED LIST ---")
    print("1. Insert Begin")
    print("2. Insert End")
    print("3. Insert at Specific Position")
    print("4. Delete Begin")
    print("5. Delete End")
    print("6. Delete at Specific Position")
    print("7. Count")
    print("8. Display")
    print("9. End")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        cll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        cll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        cll.insert_at_position(data, position)

    elif choice == 4:
        cll.delete_begin()

    elif choice == 5:
        cll.delete_end()

    elif choice == 6:
        position = int(input("Enter position: "))
        cll.delete_at_position(position)

    elif choice == 7:
        cll.count()

    elif choice == 8:
        cll.display()

    elif choice == 9:
        print("Program ended")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab4.py

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 1
Enter data: 2

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 4


--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 8
Circular Linked List: 10 -> 20 -> 30 -> HEAD

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 7
Number of nodes: 3
```

```
--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 4

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 8
Circular Linked List: 20 -> 30 -> HEAD

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 9
Program ended
```

Explanation Of Codes:

A circular linked list is a linked list where the last node points back to the first node (head).

- **Node:** Stores data and next.
- **Head:** Points to the first node.
- **Insert Begin:** Adds a node before the current head.
- **Insert End:** Adds a node after the last node.
- **Insert Position:** Inserts a node at the given position.
- **Delete Begin:** Removes the first node and updates head.
- **Delete End:** Removes the last node.
- **Delete Position:** Removes a node from the specified position.
- **Count:** Counts all nodes by traversing the list.
- **Display:** Displays nodes until it reaches head again.

- **End:** Stops the program using break.

Error Table:

S.NO	Error Obtained	Rectification
1.	Forgetting to connect the last node to head.	Set the last nodes's next = head.
2.	Deleting from an empty list	Check if self.head is None before deletion.
3.	Infinite loop during traversal.	Stop traversal when temp = = self.head.

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1. Insert at Beginning	**O(n)**	**O(1)**	Finds the last node to update its link.
2.Insert at End	**O(n)**	**O(1)**	Traverse to the last node.
3.Delete at Beginning	**O(n)**	**O(1)**	Finds the last node.
4.Delete at End	**O(n)**	**O(1)**	Traverse to the second-last node
5.Delete by value	**O(n)**	**O(1)**	Traverses to the specified position
6.Count Nodes	**O(n)**	**O(1)**	Visits every node

7.Display	**O(n)**	**O(1)**	Visits every node

WEEK-5

AIM:5a. Write a Python program to implement a stack using an array and a linked list. Implement the stack operations such as push, pop, peek, and also display the stack elements.

Program:

Stack Using Array:

```
stack = []
def push():
    element = int(input("Enter element to push: "))
    stack.append(element)
    print(element, "pushed into stack")
def pop():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print(stack.pop(), "popped from stack")
def peek():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Top element is:", stack[-1])
def display():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Stack elements are:")
        for i in reversed(stack):
            print(i)
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/
ab4.py

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 10
10 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 20
20 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 20

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 popped from stack
```

Error Table:

S.NO	Error Obtained	Rectification
1.	Trying to access stack[-1] when empty	Check whether stack is empty
2.	IndexError: pop from empty list - Trying to pop when stack is empty	Check len(stack) == 0 before pop
3.	Wrong display order - Elements displayed from bottom to top	Use reversed(stack)

Explanation Of Code:

stack = [] creates an empty list to store stack elements.

Push: append() adds a new element to the top of the stack.

Pop: pop() removes the top element from the stack.

Peek: stack[-1] displays the top element without removing it.

Display: reversed(stack) displays elements from top to bottom.

Empty check: len(stack) == 0 checks whether the stack is empty.

Menu: The while loop repeatedly displays the operations until the user selects Exit.

Stack Using Linked List:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
top = None
def push():
    global top

    element = int(input("Enter element to push: "))

    new_node = Node(element)
    new_node.next = top
    top = new_node

    print(element, "pushed into stack")
def pop():
    global top

    if top is None:
        print("Stack is empty")
    else:
        print(top.data, "popped from stack")
        top = top.next
def peek():
    if top is None:
        print("Stack is empty")
    else:
        print("Top element is:", top.data)
def display():
    if top is None:
        print("Stack is empty")
    else:
        temp = top
        print("Stack elements are:")

        while temp is not None:
            print(temp.data)
            temp = temp.next
```

```
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab 5.py

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 23
23 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
23 popped from stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack is empty

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 12
12 pushed into stack
```

Error Table:

S.NO	Error Obtained	Rectification
1.	AttributeError - Accessing top.data when top is None	Check top is None
2.	Pop does not work - top is not updated	Use top = top.next
3.	Display gives wrong result - Temporary pointer is not moved	Use temp = temp.next

Explanation of Code:

Node class creates a node containing data and a next pointer.

top = None indicates that the stack is initially empty.

Push: A new node is created and connected before the current top. The new node becomes top.

Pop: The top node is removed by making top = top.next.

Peek: top.data displays the top element without removing it.

Display: A temporary pointer temp traverses the linked list and displays every element.

Empty check: top is None checks whether the stack contains any elements.

Menu: The while loop allows the user to repeatedly perform stack operations.

Lab-6

6a. AIM: Write a Python program to implement a queue and its operations such as enqueue, dequeue, peek, and display with a menu-driven approach, implemented using an array and a linked list.

PROGRAM: Using array

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
```

```
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
```

```
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
```

```
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

output:-

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/P
RACTICE.py
Enter the size of Queue: 5

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 12
12 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
12 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty
```

ERROR TABLE

S.NO	Error Obtained	Rectification
1.	**Queue Underflow / Error when deleting from an empty queue** – front is None	Check if front is None before performing dequeue.
2.	**Rear pointer remains incorrect after deleting the last element** – rear still points to the deleted node	After dequeue, check if front is None and set rear = None.

PROGRAM: Using linked list:

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")
```

```
def dequeue(self):
    if self.front == -1:
        print("Queue Underflow")
    else:
        x = self.queue[self.front]
        self.queue[self.front] = None

        print(f"{x} deleted from the queue")
        self.front += 1

        if self.front > self.rear:
            self.front = -1
            self.rear = -1

def peek(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("Front element:", self.queue[self.front])

def display(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("The elements of the queue are:")
        for i in range(self.front, self.rear + 1):
```

```
            print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid Choice!")
```

output:-

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/P
RACTICE.py
Enter the size of Queue: 4

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 23
23 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
23 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty
```

EXPLANATION:

· class QueueEx: Creates a class named QueueEx for implementing the queue.

· __init__(self, size=5): Constructor that initializes the queue with a default size of 5.

· self.size = size: Stores the user-given queue size.

· self.queue = [None] * size: Creates a list of the given size and initialises all positions with None.

· self.front = -1: Initialises front to indicate that the queue is empty.

· self.rear = -1: Initialises rear to indicate that no element is present.

enqueue()

· if self.rear == self.size - 1: Checks whether rear has reached the last position.

· self.front = 0 : Sets the front position when inserting the first element.

· self.rear += 1 : Increases rear to the next position.

· self.queue[self.rear] = x : Stores x at the rear position.

· print() : Displays the insertion message.

dequeue()

· if self.front == -1 or self.front > self.rear: Checks whether the queue is empty.

· x = self.queue[self.front] : Stores the front element in x.

· self.queue[self.front] = None : Removes the front element from the list.

· self.front += 1 : Moves front to the next position.

· if self.front > self.rear: : Checks whether all elements have been deleted.

· self.front = -1 and self.rear = -1 : Resets the queue to the empty state.

peek()

· if self.front == -1: Checks whether the queue is empty.

· self.queue[self.front] : Accesses and displays the front element without deleting it.

display()

· if self.front == -1: Checks whether the queue is empty.

· range(self.front, self.rear + 1) : Traverses the queue from front to rear.

· print(self.queue[i]) : Displays each queue element.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Queue Overflow occurs when the queue is full.	Check if self.rear == self.size - 1 before inserting an element.
2.	Queue Underflow occurs when the queue is empty.	Check if `self.front == -1` or `self.front > self.rear` before deleting an element.

6b. AIM: Write a Python program to implement a queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach, implemented using a circular linked list.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self, size=5):
        self.size = size
        self.front = None
```

```
        self.rear = None
        self.count = 0

    def enqueue(self, x):
        if self.count == self.size:
            print("Queue Overflow")
        else:
            new_node = Node(x)

            if self.front is None:
                self.front = new_node
                self.rear = new_node
                new_node.next = new_node
            else:
                new_node.next = self.front
                self. rear.next = new_node
                self.rear = new_node

            self.count += 1
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            x = self.front.data
```

```
        if self.front == self.rear:
            self.front = None
            self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front

        self.count -= 1
        print(f"{x} deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")

            temp = self.front

            while True:
```

```
            print(temp.data)
            temp = temp.next

            if temp == self.front:
                break
size = int(input("Enter the size of Queue: "))
q = CircularQueue(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
```

```
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid Choice!")
```

OUTPUT:-

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/P
RACTICE.py
Enter the size of Queue: 5

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 45
45 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
45 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty
```

EXPLANATION:

1. Node Linking

· self.next = None : Initializes the node link as empty.

· new_node = Node(x) : Creates a new node with the given data.

· new_node.next = new_node : Makes the first node point to itself, forming a circular link.

2. Circular Connection During Enqueue

· new_node.next = self.front : Connects the new node to the first node.

· self.rear.next = new_node : Connects the current last node to the new node.

· self.rear = new_node: Updates the last node to the newly inserted node.

3. Circular Connection During Dequeue

· self.front = self.front.next : Moves front to the next node.

· self.rear.next = self.front : Reconnects the last node to the new front node.

· if self.front == self.rear: Checks whether only one node exists.

· self.front = None: Removes the only node by clearing the front reference.

· self.rear = None: Clears the rear reference.

4. Circular Traversal

· temp = self.front: Starts traversal from the first node.

· temp = temp.next : Moves to the next linked node.

· if temp == self.front: : Detects that traversal has returned to the first node.

· break : Stops traversal after one complete circular cycle.

5. Node Data Access

· self.front.data : Accesses the data stored in the front node.

· temp.data : Accesses the data of the current node during traversal.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	new_node.next is not connected to front, breaking the circular link	Set new_node.next = self.front.
2.	self.rear.next is not updated after insertion or deletion.	Update self.rear.next = new_node during enqueue and self. rear.next = self.front during deletion.

WEEK-7

AIM: 7. Write a program to implement a binary tree using an array and linked list with all of its operations.

Program:

Binary Tree Using An Array:

```
class BinaryTree:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def insert(self, value):
        for i in range(self.size):
            if self.tree[i] is None:
                self.tree[i] = value
                print(value, "inserted")
                return
        print("Tree is full")

    def search(self, value):
        for i in range(self.size):
            if self.tree[i] == value:
                print(value, "found at index", i)
                return
        print(value, "not found")

    def delete(self, value):
        for i in range(self.size):
            if self.tree[i] == value:
                self.tree[i] = None
                print(value, "deleted")
                return
        print(value, "not found")

    def inorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    def preorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def postorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")
```

```
bt = BinaryTree(10)

while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        bt.insert(value)
    elif choice == 2:
        value = int(input("Enter value to delete: "))
        bt.delete(value)
    elif choice == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)
    elif choice == 4:
        bt.inorder()
        print()
    elif choice == 5:
        bt.preorder()
        print()
    elif choice == 6:
        bt.postorder()
        print()
    elif choice == 7:
        break
    else:
        print("Invalid choice")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab 7.py

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 12
12 inserted

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 2
Enter value to delete: 23
23 not found

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 3
Enter value to search: 12
12 found at index 0

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice:
```

Explanation of Code:

1. **Initialisation**: class BinaryTree: Creates

a binary tree class.

__init__(self, size): Initializes the tree.

self.tree = [None] * size: Creates an empty

array.

self.size = size: Stores the maximum size of the array.

2. **Insertion** insert(self, value): Inserts a value into the tree.

The for loop checks each array position.

if self.tree[i] is None: Checks whether the position is empty. self.tree[i] = value: Stores the value.

return: Stops the function after insertion.

If no empty position exists, it displays Tree is full.

3. **Searching** search(self, value): Searches for a value.

The for loop checks each array element.

self.tree[i] == value: Compares the element with the search value.

If found, it displays the value and its index.

Otherwise, it displays not found.

4. **Deletion** delete(self, value): Deletes a value from the tree.

The for loop searches for the value.

self.tree[i] = None: Removes the value from that position.

If the value is not found, it displays not found.

5. **Inorder Traversal**

inorder(self, i=0): Traverses the tree in inorder.

Order: **Left → Root → Right**

2*i + 1: Finds the left child.

2*i + 2: Finds the right child.

Prints the root between the left and right subtrees.

6. **Preorder Traversal** preorder(self, i=0):

Traverses the tree in preorder.

Order: **Root → Left → Right**

Prints the root first.

Visits the left subtree.

Visits the right subtree.

7. **Postorder Traversal**

postorder(self, i=0): Traverses the tree in postorder.

Order: **Left → Right → Root**

Visits the left subtree.

Visits the right subtree.

Error Table:

S.NO	Error Obtained	Rectification
1.	Attribute Error- Accessing a node that is None.	Check whether the node exists before accessing its attribute.
2.	Tree is full- The array has no empty positions	Increase the array size
3.	Incorrect traversal output- Using the wrong traversal order or child indexes	Verify the traversal order and child indexes.

Binary Tree Using Linked List:

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = Node(data)

        if self.root is None:
            self.root = new_node
            print(data, "inserted")
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                print(data, "inserted")
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                print(data, "inserted")
                return
            else:
                queue.append(current.right)

    def search(self, data):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]

        while queue:
```

```
        while queue:
            current = queue.pop(0)

            if current.data == data:
                print(data, "found")
                return

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print(data, "not found")

    def delete(self, data):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]
        target = None
        deepest = None
        parent = None
        deepest_parent = None

        while queue:
            current = queue.pop(0)

            if current.data == data:
                target = current

            if current.left:
                parent = current
                queue.append(current.left)
                deepest_parent = current

            if current.right:
                parent = current
                queue.append(current.right)
                deepest_parent = current

            deepest = current

        if target is None:
            print(data, "not found")
        if target is None:
            print(data, "not found")
            return

        if deepest == self.root:
            self.root = None
        else:
            if deepest_parent.left == deepest:
                deepest_parent.left = None
            else:
                deepest_parent.right = None

            target.data = deepest.data

        print(data, "deleted")

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")
```

```
bt = BinaryTree()

while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        bt.insert(value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        bt.delete(value)

    elif choice == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)

    elif choice == 4:
        bt.inorder(bt.root)
        print()

    elif choice == 5:
        bt.preorder(bt.root)
        print()

    elif choice == 6:
        bt.postorder(bt.root)
        print()

    elif choice == 7:
        break

    else:
        print("Invalid choice")
```

Output:

```
Enter choice:
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab 7.py

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 23
23 inserted

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 2
Enter value to delete: 12
12 not found

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 3
Enter value to search: 45
45 not found

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 4
23

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 5
23

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 6
23
```

Explanation Of Code:

1. **Node Creation** class Node:

Creates a node class. self.data = data:

Stores the node's value. self.left = None:

Initialises the left child. self.right = None:

Initialises the right child.

2. **Tree Initialisation** class BinaryTree:

Creates the binary tree class.

__init__(self): Initializes the tree.

self.root = None: Indicates that the tree is initially empty.

3. **Insertion** insert(self, data): Inserts a

new node.

Node(data): Creates a new node.

if self.root is None: Checks whether the tree is empty.

If empty, the new node becomes the root.

queue = [self. root]: Creates a queue containing the root.

queue.pop(0): Removes the first node from the queue.

current. left is None: Checks whether the left child is empty.

current. right is None: Checks whether the right child is empty.

Inserts the new node at the first available position.

4. **Searching** search(self, data): Searches

for a value.

Checks whether the tree is empty.

Creates a queue starting with the root.

Checks each node's data.

Adds existing left and right children to the queue.

Displays found if the value exists.

Otherwise, displays not found.

5. Deletion

delete(self, data): Deletes a node.

Checks whether the tree is empty.

Finds the node containing the target value.

Finds the deepest, rightmost node.

Copies the deepest node's value into the target node.

Removes the deepest node from its parent.

If the root is the only node, sets self. root = None.

6. Inorder Traversal

inorder(self, node): Traverses the tree in

inorder.

Order: **Left → Root → Right**

Visits the left child.

Prints the current node.

Visits the right child.

7. Preorder Traversal preorder(self,

node): Traverses the tree in preorder.

Order: **Root → Left → Right**

Prints the current node first.

Visits the left child.

Visits the right child.

Error Table:

S.NO	Error Obtained	Rectification
1.	Type Error- Using an invalid data Type.	Convert input using int( )
2.	Value not found- Searching for a value that does not exist.	Enter an existing value.
3.	Tree is full- The array has no empty positions	Increase the array size.

WEEK-8

AIM: 8. Write a Python program to implement Binary Search Trees with its operations.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)

        return root

    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)
        else:
            return self.search(root.right, key)

    def find_min(self, root):
        while root.left is not None:
            root = root.left
        return root
```

```
    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)
        else:
            return self.search(root.right, key)

    def find_min(self, root):
        while root.left is not None:
            root = root.left
        return root
```

```
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.data:
            root.left = self.delete(root.left, key)
```

```
                elif root.right is None:
                    return root.left

                temp = self.find_min(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)

            return root


        def inorder(self, root):
            if root:
                self.inorder(root.left)
                print(root.data, end=" ")
                self.inorder(root.right)

        def preorder(self, root):
            if root:
                print(root.data, end=" ")
                self.preorder(root.left)
                self.preorder(root.right)

        def postorder(self, root):
            if root:
                self.postorder(root.left)
                self.postorder(root.right)
                print(root.data, end=" ")


tree = BST()

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter value to insert: "))
        tree.root = tree.insert(tree.root, data)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        result = tree.search(tree.root, key)

        if result:
            print("Element found.")
        else:
            print("Element not found.")

    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(tree.root, key):
            tree.root = tree.delete(tree.root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(tree.root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(tree.root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(tree.root)
```

```
    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab 8.py


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 12
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 2
Enter value to search: 12
Element found.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
```

Explanation Of Code:

1. Node Class: Creates a node with data, left child, and right child.
2. BST Class: Creates a Binary Search Tree with the root initially set to None.
3. Insertion: The insert() function adds a new node in the correct position. Smaller values go to the left, and greater values go to the right.
4. Searching: The search() function checks whether the given element exists in the tree.

5. Finding Minimum: The find_min() function finds the smallest value by moving towards the leftmost node.

6. Deletion: The delete() function removes a node while maintaining the BST properties. It handles nodes with no children, one child, or two children.

7. Inorder Traversal: The inorder() function visits the left subtree, root, and right subtree. It displays the values in ascending order.

8. Preorder Traversal: The preorder() function visits the root, left subtree, and right subtree.

9. Postorder Traversal: The postorder() function visits the left subtree, right subtree, and root.

10. Main Program: Creates a BST object and displays a menu for insertion, searching, deletion, traversals, and exit.

11. While Loop: Repeats the menu until the user chooses the exit option.

Error Table:

S.NO	Error Obtained	Rectification
1.	ValueError – Entering a non-integer value when an integer is expected	Enter a valid integer.
2.	RecursionError – Excessive calls for a deeply unbalanced tree.	Use an iterative approach or a balanced BST.
3.	Duplicate values ignored – The insertion method does not insert duplicate values.	Enter a unique value.

Time Complexity:

Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Searching	O(log n)	O(n)
Traversal	O(n)	O(n)
Deletion	O(log n)	O(n)

WEEK-9

AIM: 9. Write a Python program to implement AVL trees with its operations.

Program:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:
    def height(self, root):
        if root is None:
            return 0
        return root.height

    def get_balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        return x

    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        return y
```

```
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)
```

```
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def search(self, root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    def find_min(self, root):
        while root.left is not None:
            root = root.left
        return root

    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)


        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root
```

```
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

tree = AVLTree()
root = None

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Element found.")
        else:
            print("Element not found.")

    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(root)

  elif choice == 7:
      print("Program terminated.")
      break

  else:
      print("Invalid choice.")
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/
ab 9.py


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 45
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 2
Enter value to search: 23
Element not found.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 3
Enter value to delete: 45
Node deleted successfully.
```

Explanation Of Code:

1. Node Class: Creates a node containing a key, left child, right child, and height.
2. Height: Returns the height of a node.
3. Balance Factor: Calculates the difference between the heights of the left and right subtrees.
4. Right Rotation: Balances a tree when the left side becomes too heavy.
5. Left Rotation: Balances a tree when the right side becomes too heavy.

6. Insertion: Inserts a new value and balances the tree if necessary.

7. Searching: Finds whether a given value exists in the tree.

8. Finding Minimum: Finds the smallest value in a subtree.

9. Deletion: Removes a node and restores the AVL balance.

10. Inorder Traversal: Visits Left → Root → Right and displays values in ascending order.

11. Preorder Traversal: Visits Root → Left → Right.

12. Postorder Traversal: Visits Left → Right → Root.

13. Main Program: Displays a menu to perform operations until the user chooses Exit.

Four Rotation Cases:

Case	Condition	Rotation
LL(Left-Left)	Left subtree becomes too heavy on its left side	Right rotation
RR(Right-Right)	Right subtree becomes too heavy on its right side	Left Rotation
LR(Left-Right)	Left subtree becomes too heavy on its right side	Left rotation, then right rotation
RL(Right-Left)	Right subtree becomes too heavy on its left side	

Error Table:

S.NO	Error Obtained	Rectification
1.	ValueError – Entering a non-integer	Enter a valid integer.
2.	AttributeError – Accessing a property of None	Check whether the node exists
3.	Element not found – Value does not exist in the tree	Enter an existing value.

Time Complexity:

Operation	Time Complexity
Insertion	O(log n)
Searching	O(log n)
Traversal	O(n)
Deletion	O(log n)
Rotation	O(1)

WEEK-10

AIM: 10. Write a Python program to Implement Hashing Using Separate Chaining, Linear Probing and Quadratic Probing.

Program:

```python
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for i in range(size)]

    def insert(self, key):
        index = key % self.size
        self.table[index].append(key)

    def display(self):
        print("Separate Chaining:")
        for i in range(self.size):
            print(i, ":", self.table[i])

class LinearProbing:
    def __init__(self, size):
        self.size = size
        self.table = [-1] * size

    def insert(self, key):
        index = key % self.size

        while self.table[index] != -1:
            index = (index + 1) % self.size

        self.table[index] = key

    def display(self):
        print("\nLinear Probing:")
        for i in range(self.size):
            print(i, ":", self.table[i])

class QuadraticProbing:
    def __init__(self, size):
        self.size = size
        self.table = [-1] * size

    def insert(self, key):
        index = key % self.size
        i = 0

        while self.table[(index + i * i) % self.size] != -1:
            i += 1

        self.table[(index + i * i) % self.size] = key

        self.table[(index + i * i) % self.size] = key

    def display(self):
        print("\nQuadratic Probing:")
        for i in range(self.size):
            print(i, ":", self.table[i])


size = 7
keys = [10, 20, 15, 7, 22]

h1 = HashTable(size)
h2 = LinearProbing(size)
h3 = QuadraticProbing(size)

for key in keys:
    h1.insert(key)
    h2.insert(key)
    h3.insert(key)

h1.display()
h2.display()
h3.display()
```

Output:

```
= RESTART: C:/Users/bijiv/AppData/Local/Programs/Python/Python313/l
ab 10.py
Separate Chaining:
0 : [7]
1 : [15, 22]
2 : []
3 : [10]
4 : []
5 : []
6 : [20]

Linear Probing:
0 : 7
1 : 15
2 : 22
3 : 10
4 : -1
5 : -1
6 : 20

Quadratic Probing:
0 : 7
1 : 15
2 : 22
3 : 10
4 : -1
5 : -1
6 : 20
```

Explanation Of Code:

1. **Import and Class Creation**: class HashTable: creates a class to implement separate chaining.
2. def __init__(self, size): initializes the hash table with the given size.

   self.size = size stores the table size.

   self.table = [[] for i in range(size)] creates an empty list at every index.
3. **Separate Chaining – Insert Function** def insert(self, key): inserts a key into the hash table. index = key % self.size calculates the hash index.

   self.table[index].append(key) inserts the key into the list at that index.

If two keys have the same index, both are stored in the same list.

4. **Separate Chaining – Display Function** def display(self): displays the hash table. for i in range(self.size): visits every index.

print(i, ":", self.table[i]) prints the index and its values.

5. **Linear Probing** class LinearProbing: creates a class for linear probing. self.table = [-1] * size creates an empty hash table using -1. index = key % self.size calculates the hash index. while self.table[index] != -1: checks whether the position is occupied. index = (index + 1) % self.size checks the next position if a collision occurs.

self.table[index] = key inserts the key into the available position.

The display() function prints all table elements.

6. **Quadratic Probing** class QuadraticProbing: creates a class for quadratic probing.

self.table = [-1] * size initializes the hash table. index = key % self.size calculates the initial index. i = 0 initialises the attempt counter.

while self.table[(index + i * i) % self.size] != -1: checks whether the calculated position is occupied.

i += 1 increases the attempt counter.

self.table[(index + i * i) % self.size] = key inserts the key into the available position.

The display() function displays the hash table.

7. **Main Program**

size = 7 sets the hash table size to 7. keys = [10, 20, 15, 7, 22] stores the input keys. h1, h2, and h3 create objects for the three hashing techniques.

The for loop inserts each key into all three hash tables.

Finally, display() prints the results of all three techniques.

Error Table:

S.NO	Error Obtained	Rectification
1.	Collision Error – Two keys get the same index	Use collision handling techniques
2.	AttributeError – Calling an undefined method	Check class and method names
3.	Infinite Loop – Hash Table is full	Check for empty positions.

Time Complexity And Space Complexity:

Hashing Technique	Best Time	Average Time	Worst Time	Space Complexity
Separate Chaining	O(1)	O(1)	O(n)	O(n+m)
Linear Probing	O(1)	O(1)	O(n)	O(m)
Quadratic Probing	O(1)	O(1)	O(n)	O(m)